Author: Lude Bakema
LabVIEW developer

Department: Test and verification engineering (PInS)

Approval

Role
R&D: Department Manager SW
R&D: System Designer
Q&R: Dev QA Officer

(approval via electronic signature in document management tool)

Revision History

Revision Date	Author	Source	Changes/Comments
2022 April 26	Lude Bakema		Rev 0.1 (Initial version)

<List the changes, date and author related to the ‘Source’. The ‘Source’ refers to the reason for the change.>

Open Issues

Table of Contents

Approval 1
Revision History 2
Open Issues 2
Table of Contents 3
1. Document Introduction 5
1.1. Purpose 5
1.2. Scope 5
1.3. References 5
1.4. Definitions & Abbreviations 5
2. Posterior coil 6
3. LabVIEW classes 6
3.1. Analyze class 7
3.1.1. Example 7
3.1.2. Main 8
3.1.3. init coils 8
3.1.4. Get coil 9
3.1.5. Put coil 9
3.1.6. Sort coil array 10
3.1.7. Get mrd file names 10
3.1.8. Process snr 11
3.1.9. Store snr conditions 11
3.1.10. Process noise 12
3.1.11. Store noise conditions 13
3.1.12. Save file 13
3.1.13. Format (de)tune data 14
3.2. Coil channel class 14
3.2.1. Init channel 14
3.2.2. Load parameters 15
3.2.3. Store channel data 16
3.2.4. Cleanup data 17
3.2.5. Amp 17
3.2.6. Amp_dB 18
3.2.7. Mean 19
3.2.8. SD_dB 19
3.2.9. M_SD 20
3.3. MRD file 20
3.3.1. MRD file specification 20
3.3.2. Init 21
3.3.3. Read 21
3.4. MRD data 23
3.4.1. Init 23
3.4.2. Write 24
3.4.3. Append 24
3.4.4. Read 25
3.4.5. Data size 25
3.4.6. Delete 26
3.4.7. Read section 26
4. Pickering power and rf switcher 28
4.1. Architectcture 28
4.2. Example 29
4.3. Read xml config 29
4.4. Connect 31
4.5. Disconnect 31
4.6. Open cards 32

4.7. Get type nrs from storage 32
4.8. Close cards 33
4.9. Get module info 33
4.10. Write power switch 34
4.11. Read power switch 35
4.12. Write rf switch 35
4.13. Read rf switch 36
4.14. Reset rf switch 36
Control of the Form 38
History of the Form 38

1. Document Introduction

1.1. Purpose

This document describes the modules and drivers which are used in the MR SNR tool for noise measurement, hereafter mentioned as SNR tool. This tool will be used in the factory Best to measure the signal to noise performance of the posterior coils during production.

1.2. Scope

This Software unit design document (SUD) is applicable to:

Software tool name	MR SNR tool for noise measurement, hereafter mentioned as SNR tool

1.3. References

Reference	Identification	Title / additional remarks

1.4. Definitions & Abbreviations

Term	Description
DUT	Device Under Test
GUI	Graphical user interface
Posterior coil	Coil in MR setup positioned under the patient inside the patient table

2. Posterior coil

This is the coil section which is located on the patient table.

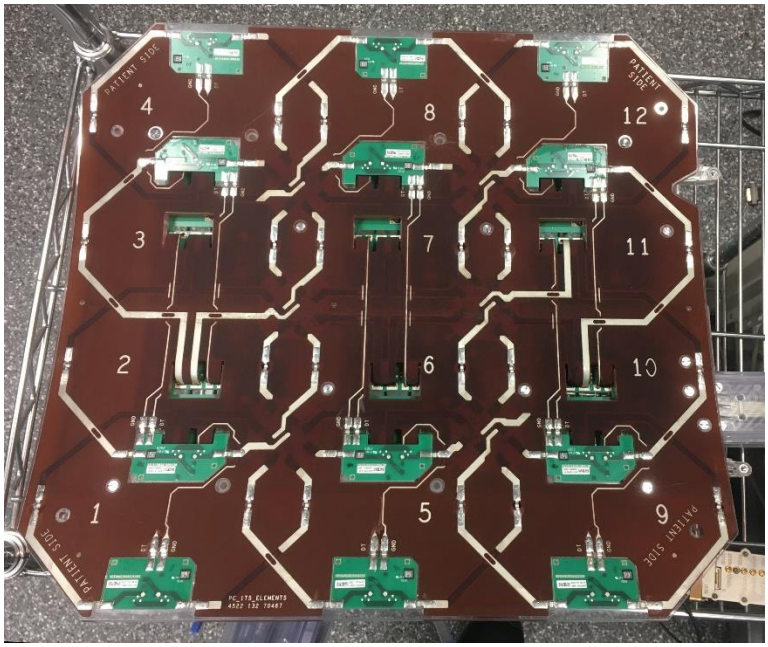


Figure 1 Posterior coil

12 sections can be identified. The data for the sections is grouped in one singe snr file as shown below:

4	8	12
3	7	11
2	6	10
1	5	9

Grouping if the coils

3. LabVIEW classes

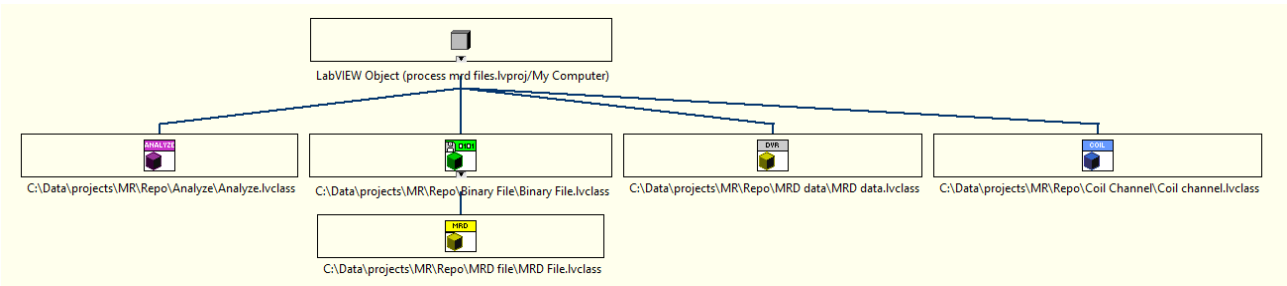


Figure 2: LabVIEW class hierarchy

Item	Description
Analyze	Analyze module for mrd data
Binary File	Parent class for reading binary files
MRD file	Child class for reading MRD files
MRD data	Storage container (DVR) for MRD data
Coil Channel	All calculations and support functions to process the coil data

3.1. Analyze class

Implementation is based on the script file: tbx_poco_snr_dB_1T5.ana

3.1.1. Example

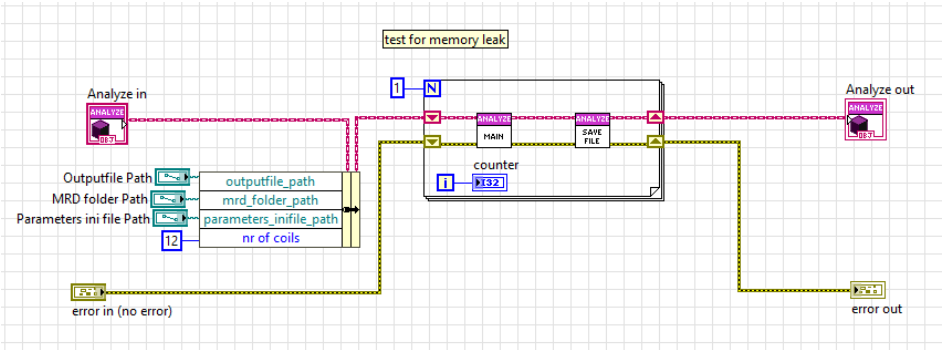


Figure 3: Block diagram of example.vi

This example runs the entire process from reading the mrd files, doing all calculation and save the results to a file.

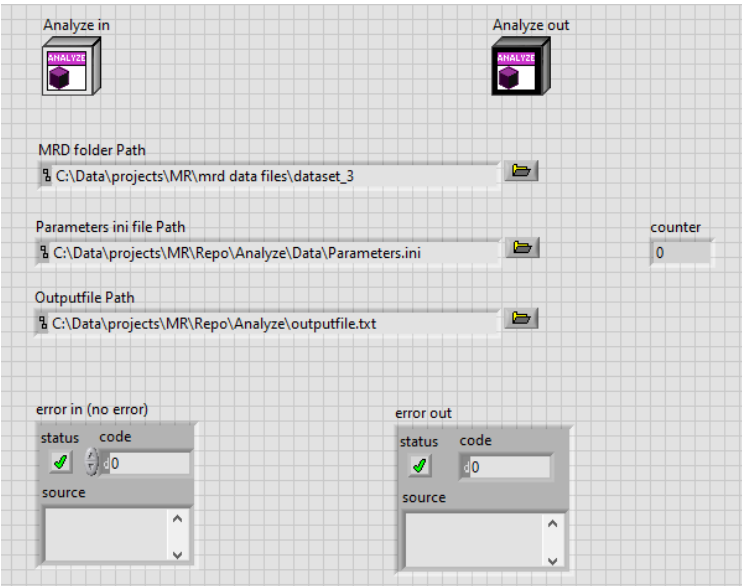


Figure 4: Front panel example.vi with default settings

Four parameters need to be set to properly run the example.

Item	Description
Output file path	Output file path for result text file
Mrd folder path	Mrd folder were the data set can be found. It should consist of 4 snr files and 1 noise file
Parameters file ini path	Parameterfile ini path were were the parameter file can be found
Nr of coils	Nr of coils used. This should be 12, but can be changed in the future when needed

The whole program is placed in a 'for loop' to run it multiple times, testing it for memory leaks.

3.1.2. Main

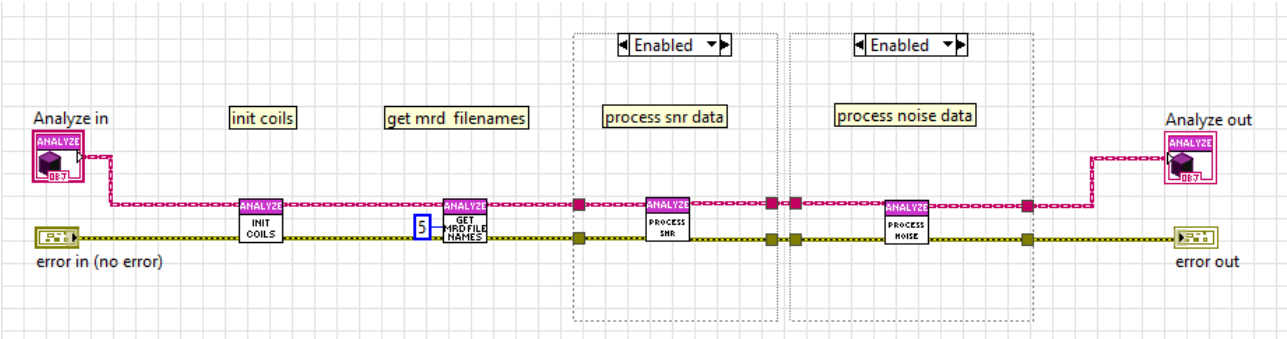


Figure 5: Block diagram main.vi

Item	Description
Init coils	Initializes a class instance for every coil section (12)
Get mrd file names	Get mrd file names from dataset directory
Process snr	Process snr data based on snr files
Process noise	Process noise data based on noise file

3.1.3. init coils

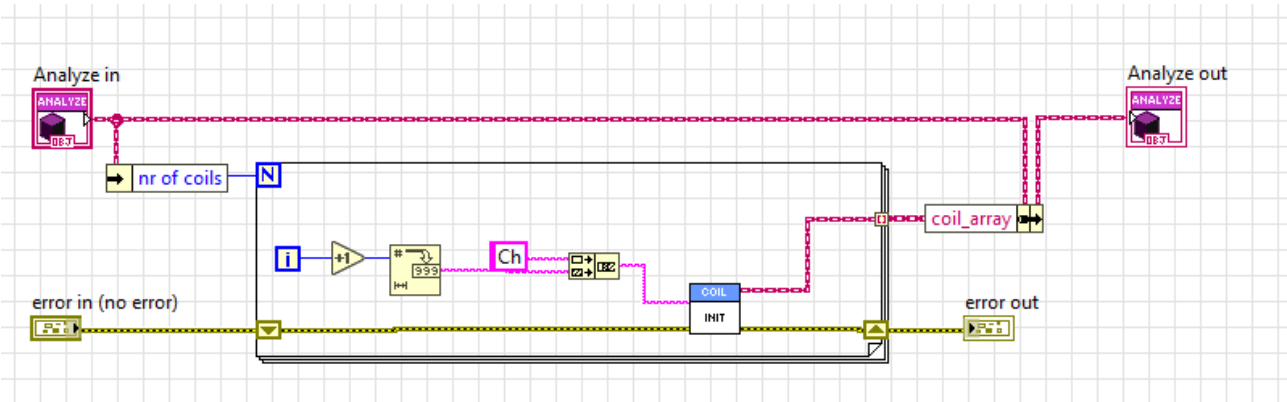


Figure 6: block diagram of init coils.vi

This sub module initializes several coil instances (12) identifiable with their own ID (Ch1, Ch2, ….).

3.1.4. Get coil

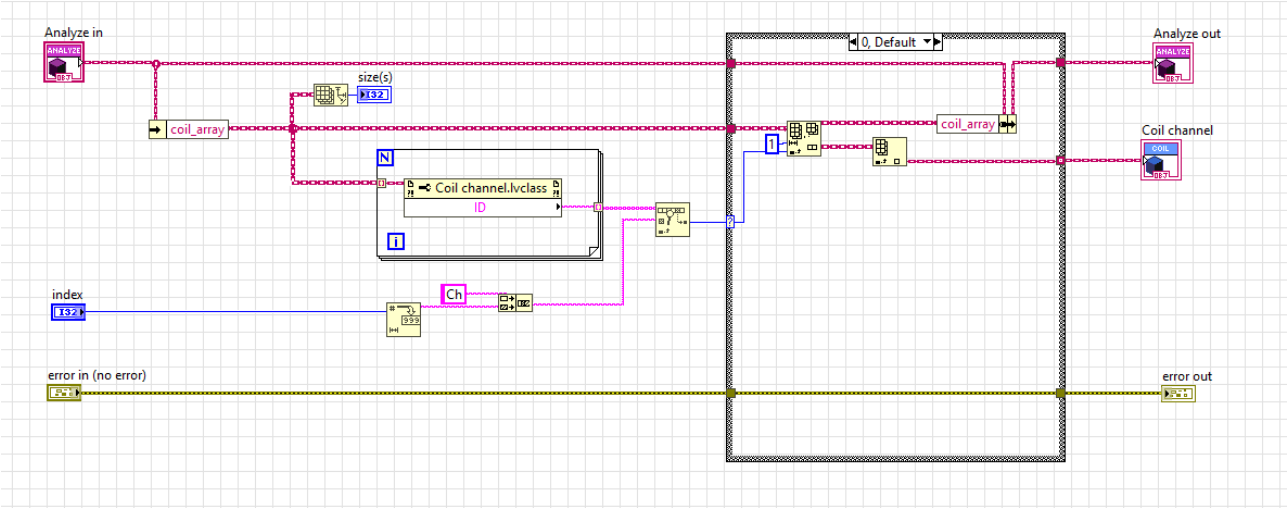


Figure 7: block diagram of get coil.vi

Gets coil based on ID and deletes it from the coil array.

3.1.5. Put coil

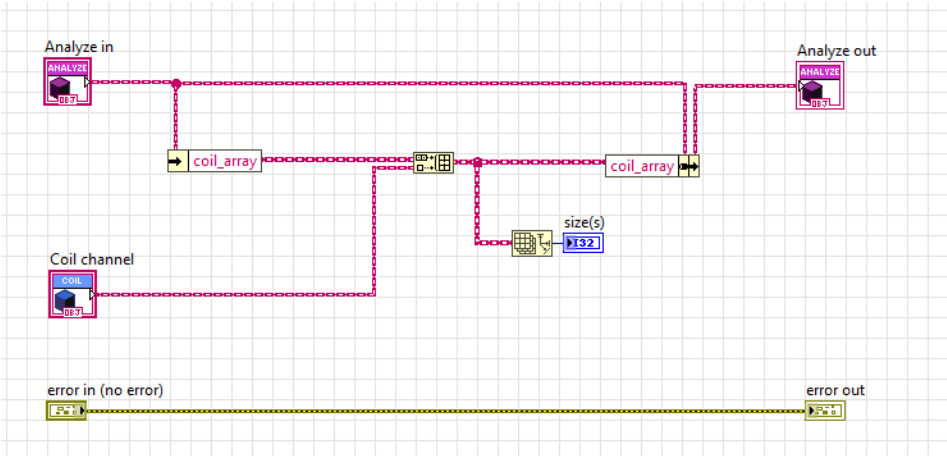


Figure 8: block diagram of put coil vi

Adds the coil instance to the coil array

3.1.6. Sort coil array

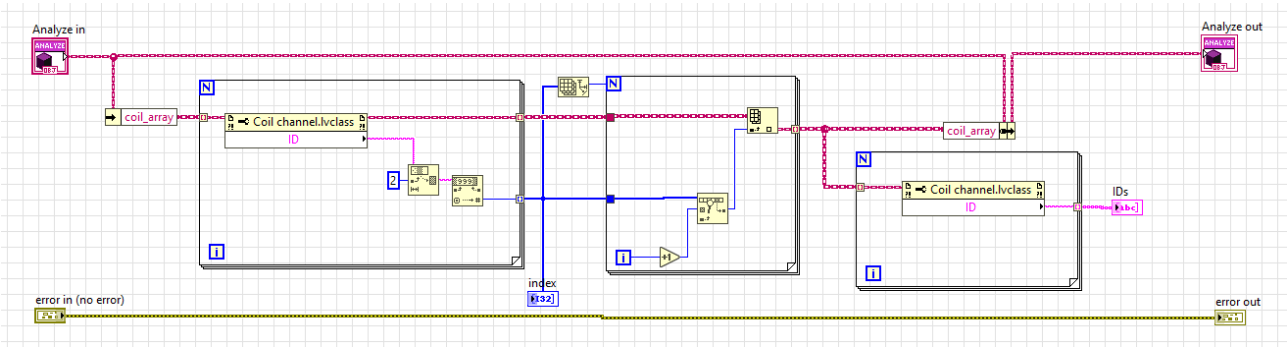


Figure 9: block diagram of sort array.vi

This function ensures that the coil instances are in a particular order 1,2,3,4…

3.1.7. Get mrd file names

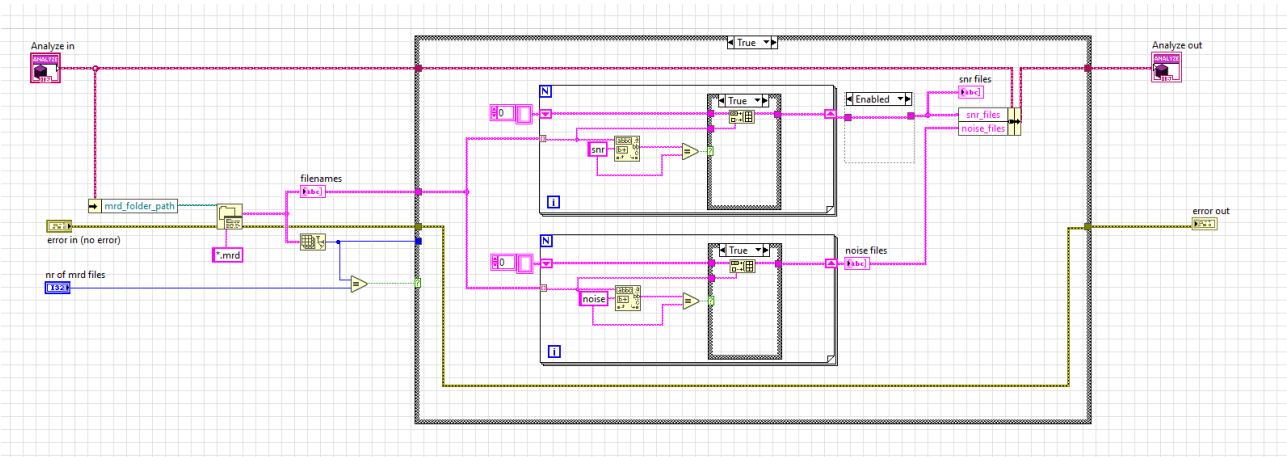


Figure 10 block diagram of get mrd file names

The mrd file names are loaded from the directory specified in mrd_folder_path and stored into 2 arrays:

- snr_files (4 files)
- noise_files (1 file)

3.1.8. Process snr

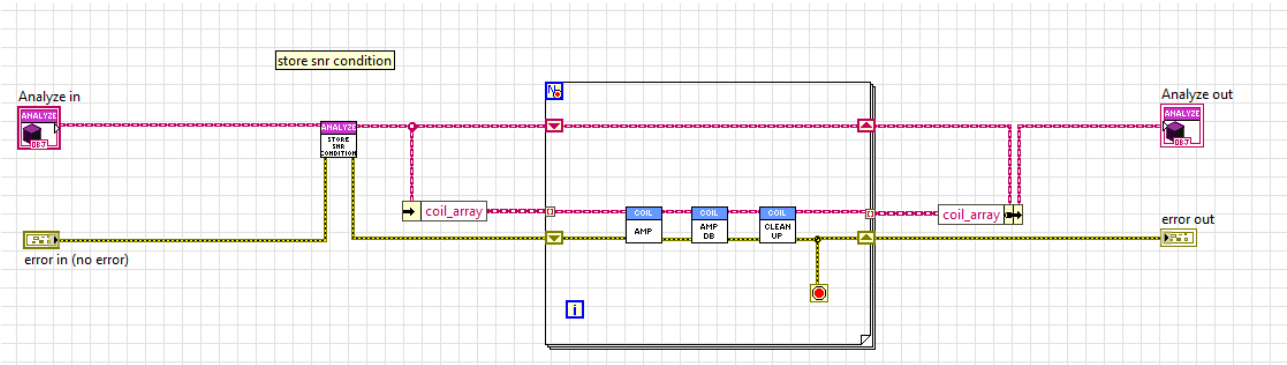


Figure 11: block diagram of process snr.vi

Amp value and amp db value is calcultated. The cleanup method deletes the mrd data from the coil channel to release the data memory.

Nr	Item	In between variable	calculation
1	Amp_tune	Ch*x*_tune	AVERAGE(COPY(Ch*x*,1,10000))
2	Amp_tune_dB	Ch*x*_tune_dB	10 * LOG((Ch*x*_tune * a*x* - b*x*)^2)
3	Amp_detune	Ch*x*_detune	AVERAGE(COPY(Ch1,10001,10000))
4	Amp_detune_dB	Ch*x*_detune_dB	10 * LOG((Ch*x*_detune * ax - bx)^2)

Note: variables names are corresponding with the original variables in the ana script
'x': coil channel nr
'a', 'b': parameters from parameter file

3.1.9. Store snr conditions

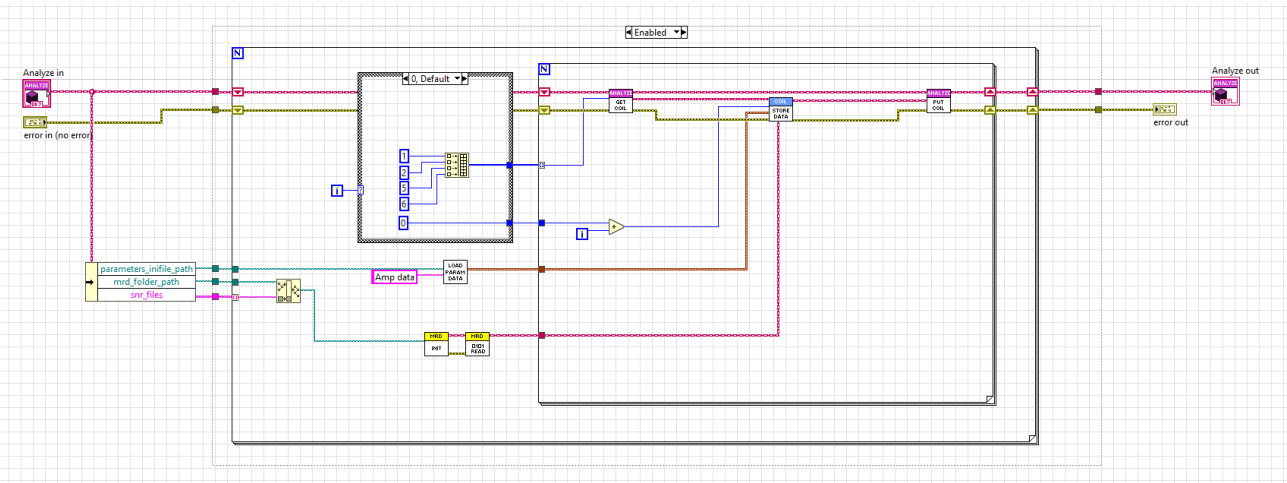


Figure 12: blok diagram of store snr conditions.vi

Every snr file is read and a subset of the mrd data is stored in the appropriate coil channel instance.
The data retrieving parameters are loaded from the ini file. In the table below is shown which file and channel nr belongs to each channel instance.

Coil instance nr	MRD file	Channel nr
1	pc_snr_1t5_1_*.MRD	0
2	pc_snr_1t5_1_*.MRD	1
5	pc_snr_1t5_1_*.MRD	2
6	pc_snr_1t5_1_*.MRD	3
3	pc_snr_1t5_2_*.MRD	0
4	pc_snr_1t5_2_*.MRD	1
7	pc_snr_1t5_2_*.MRD	2
8	pc_snr_1t5_2_*.MRD	3
9	pc_snr_1t5_3_*.MRD	2
10	pc_snr_1t5_3_*.MRD	3
11	pc_snr_1t5_4_*.MRD	2
12	pc_snr_1t5_4_*.MRD	3

Note: channel 0,1 are not used in pc_snr_1t5_3_*.MRD and pc_snr_1t5_4_*.MRD.

3.1.10. Process noise

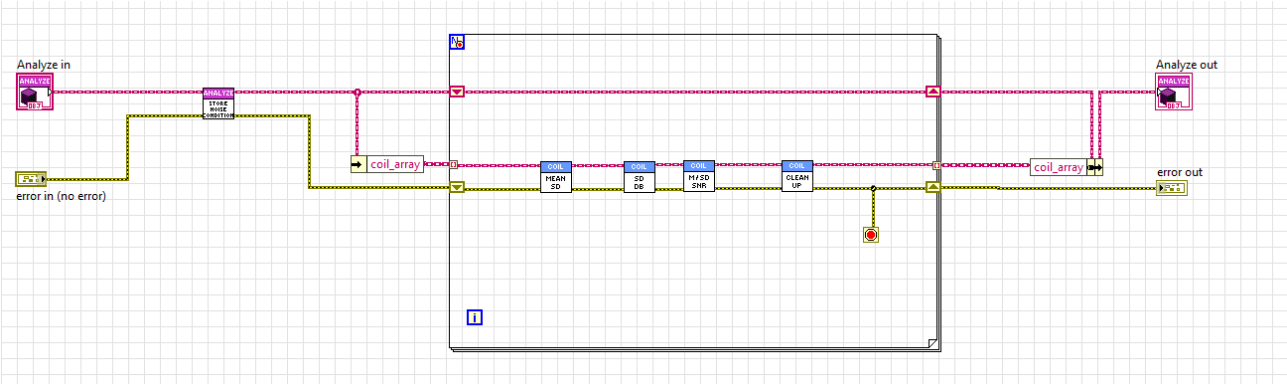


Figure 13: block diagram of process noise.vi

Mean, SD (standard deviation) is calculated. M/SR and SNR is calculated. See the table below.
The cleanup method deletes the mrd data from the coil channel to release the data memory.

Nr	Item	In between variable	calculation
1	Mean_tune	Meanx_tune = Stat_Chx_tune[2]	STATISTICS(COPY(Stat_Chx,1,262144))
2	SD_tune	Stdx_tune = Stat_Chx_tune[4]	STATISTICS(COPY(Stat_Chx,1,262144))
3	SD_tune_dB	Stdx_tune_dB	10 * LOG((Stat_Chx_tune[4] * sx - tx)^2) - Att
4	M/SD_tune	MStdx_tune = Meanx_tune / Stdx_tune	Stat_Chx_tune[2] / Stat_Chx_tune[4]
5	SNR_tune	SNRx_tune = Chx_tune / Stdx_tune	
6	Mean_detune	Meanx_detune = Stat_Chx_detune[2]	STATISTICS(COPY(Stat_Chx,262145,262144))
7	SD_detune	Stdx_detune = Stat_Chx_detune[4]	STATISTICS(COPY(Stat_Chx,262145,262144))
8	SD_detune_dB	Stdx_detune_dB	10 * LOG((Stat_Chx_tune[4] * sx - tx)^2) - Att
9	M/SD_detune	MStdx_tune = Meanx_detune / Stdx_detune	Stat_Chx_tune[2] / Stat_Chx_tune[4]
10	SNR_detune	SNRx_detune= Chx_detune / Stdx_detune	

Note: variables are corresponding with the original variables in the ana script
'x': coil channel nr
's', 't', att' = parameters from parameter file

For every coil instance, the calculations are performed and stored in the class object.

3.1.11. Store noise conditions

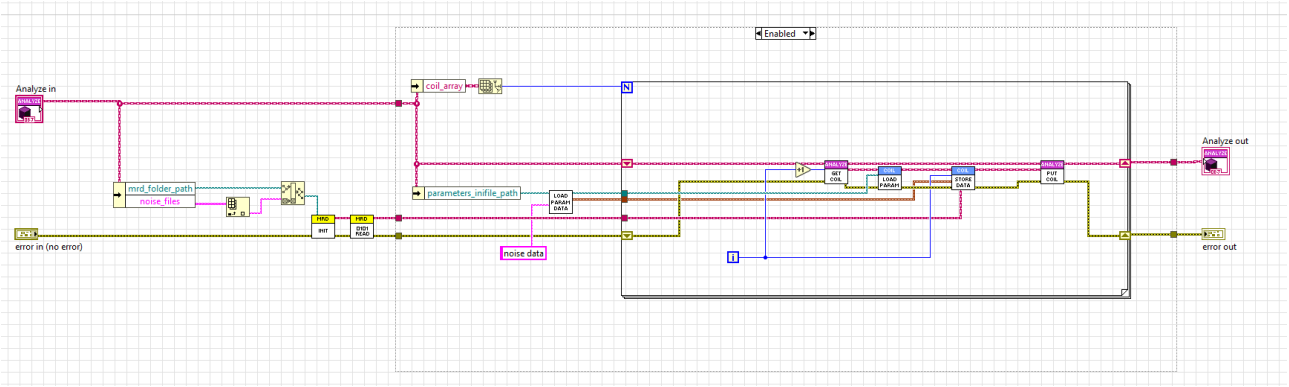


Figure 14: block diagram of store noise conditions.vi

3.1.12. Save file

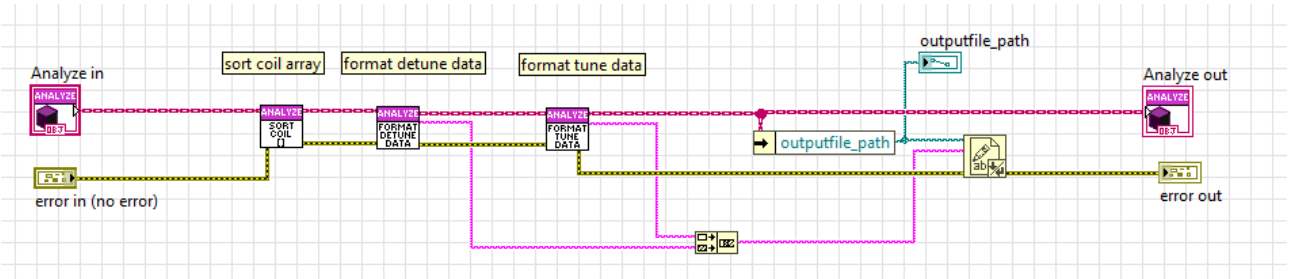


Figure 15: block diagram of save file.vi

Saves the calculated data to a text file. An example of the file is shown below:

Amp_tune= 2870780,2977950,2819169,2910595,2801148,2893577,2803559,2944975,2835417,3028758,2850869,2981916,
Amp_tune_dB= -35.26,-34.57,-35.55,-35.65,-36.00,-35.49,-35.11,-35.41,-35.39,-35.08,-35.46,-35.25,
Mean_tune= 3202,2938,3849,2746,3300,3098,3529,2884,2994,3502,3952,2881,
SD_tune= 1666,1528,2009,1429,1723,1613,1838,1500,1563,1828,2058,1499,
SD_tune_dB= -152.26,-152.61,-150.36,-153.32,-151.10,-151.53,-149.85,-152.49,-152.39,-151.78,-150.56,-153.19,
M/SD_tune= 1.922492,1.923369,1.916175,1.922102,1.915485,1.920358,1.919706,1.923088,1.915319,1.916253,1.920402,1.922243,
SNR_tune= 1723.517700,1949.531073,1403.548785,2037.171131,1625.945418,1793.730411,1525.068951,1963.764318,1813.826697,1657.306083,1385.242535,1989.535667,
Amp_detune= 314202,277914,176637,503523,199074,366078,182011,207944,182910,419686,189907,193549,
Amp_detune_dB= -54.56,-55.36,-59.88,-50.92,-59.04,-53.20,-59.10,-58.39,-59.19,-52.30,-58.99,-59.28,
Mean_detune= 11243,12888,10633,14679,11794,14413,10624,15961,10843,15761,12136,15726,
SD_detune= 5836,6752,5552,7613,6147,7524,5557,8304,5664,8182,6329,8195,
SD_detune_dB= -140.85,-139.07,-141.23,-137.99,-139.59,-137.34,-139.72,-137.11,-140.85,-138.40,-140.52,-137.90,
M/SD_detune= 1.926440,1.908888,1.915343,1.928038,1.918553,1.915488,1.911763,1.922026,1.914491,1.926187,1.917561,1.918894,
SNR_detune= 53.835237,41.162742,31.816532,66.136860,32.383381,48.653101,32.752791,25.040761,32.295133,51.292123,30.005800,23.616840,

3.1.13. Format (de)tune data

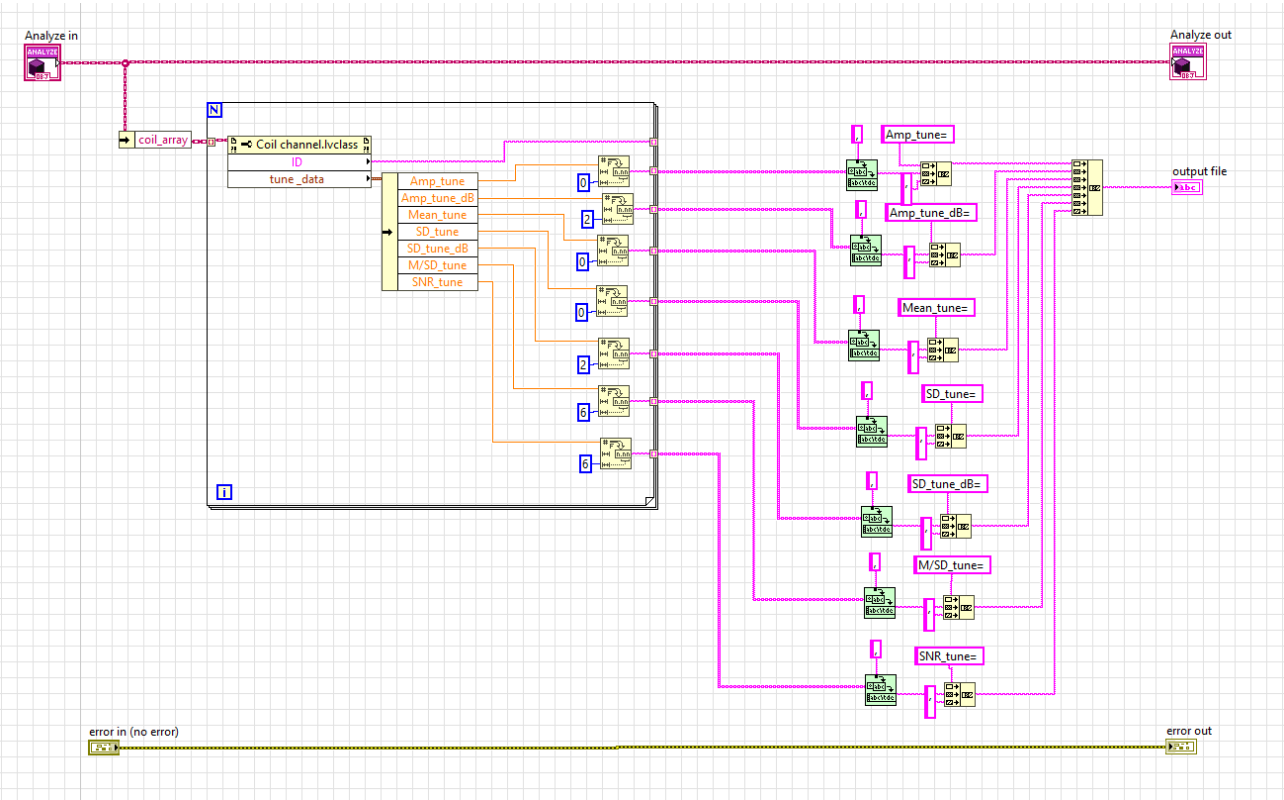


Figure 16: Block diagram of format tune data.vi

This method selects the data values from the cluster for each coil instance and put them into a text array. Format detune data.vi is similar

3.2. Coil channel class

3.2.1. Init channel

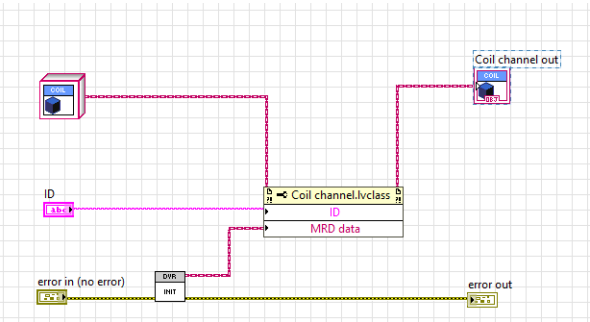


Figure 17: block diagram of init channel

Coil channel instance is created. Mrd data memory is initialized. Id is stored.

3.2.2. Load parameters

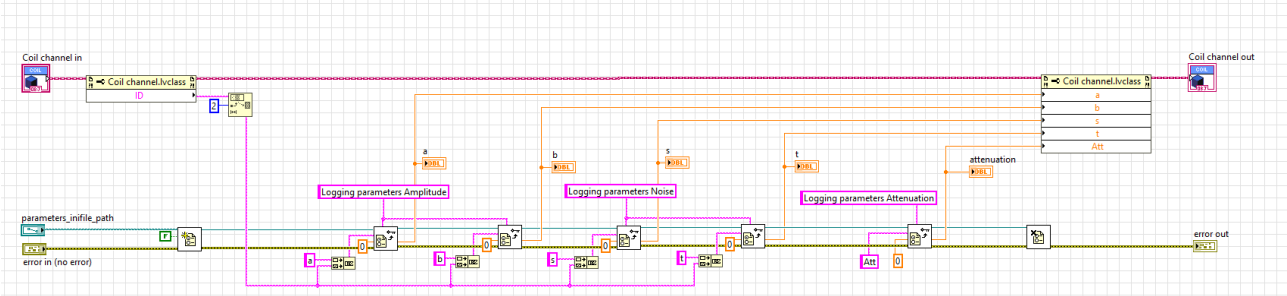


Figure 18: block diagram of load parameters.vi

For every coil channel there are calibration constants stored in the parameter file. Please find an example below. Every coil channel loads its specific constants and stores the values in the class.

[Logging parameters Amplitude]
a1 = 6.01884E-9
a2 = 6.28655E-9
a3 = 5.93261E-9
a4 = 5.67390E-9
a5 = 5.66251E-9
a6 = 5.78158E-9
a7 = 6.27387E-9
a8 = 5.75843E-9
a9 = 5.99962E-9
a10 = 5.82198E-9
a11 = 5.91867E-9
a12 = 5.81035E-9

b1 = 1.94550E-5
b2 = 4.15348E-5
b3 = 3.45843E-5
b4 = 1.16375E-5
b5 = 1.03068E-5
b6 = -7.03862E-5
b7 = 3.28509E-5
b8 = -6.06690E-6
b9 = -2.41626E-7
b10 = 1.79733E-5
b11 = 4.85375E-7
b12 = 3.80999E-5

[Logging parameters Noise]
s1 = 6.38403E-10
s2 = 6.76440E-10
s3 = 6.39607E-10
s4 = 6.78481E-10
s5 = 6.98680E-10
s6 = 7.42876E-10
s7 = 7.68253E-10
s8 = 6.83088E-10
s9 = 6.53492E-10
s10 = 5.97533E-10
s11 = 6.06854E-10

s12 = 6.32669E-10

t1 = 8.37241E-8
t2 = 9.29840E-8
t3 = 6.52347E-8
t4 = 1.02776E-7
t5 = 8.43579E-8
t6 = 1.33505E-7
t7 = 1.19060E-7
t8 = 7.01360E-8
t9 = 5.65227E-8
t10 = 5.70292E-8
t11 = 5.79068E-8
t12 = 6.85406E-8

[Logging parameters Attenuation]
Att = 32.08

3.2.3. Store channel data

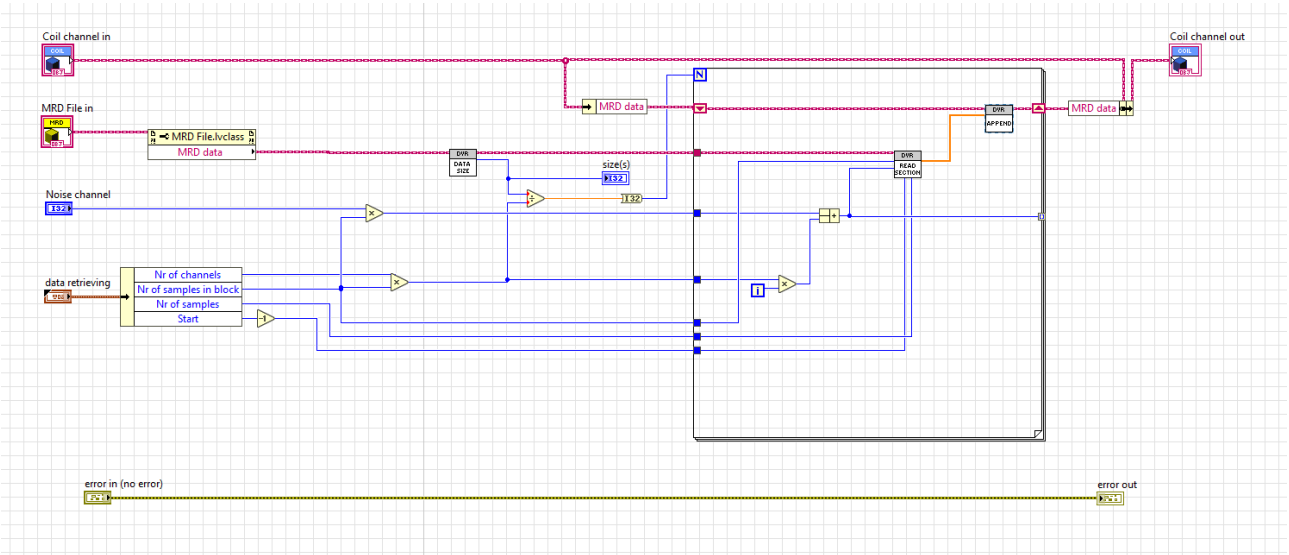


Figure 19: block diagram of store channel.vi

This method read the data blocks for each channel from the mrd data file. It appends the blocks so that one array of data is generated to be used for further calculations. Data retrieving parameters are loaded from the parameter file. The values are different depending on the type of mrd file(snr file or noise file), See an example from the parameter file below.

[snr data]
Start = 412
Nr of samples = 200
Nr of samples in block = 1024
Nr of channels = 4

[noise data]
Start = 1
Nr of samples = 1024
Nr of samples in block = 1024
Nr of channels = 12

3.2.4. Cleanup data

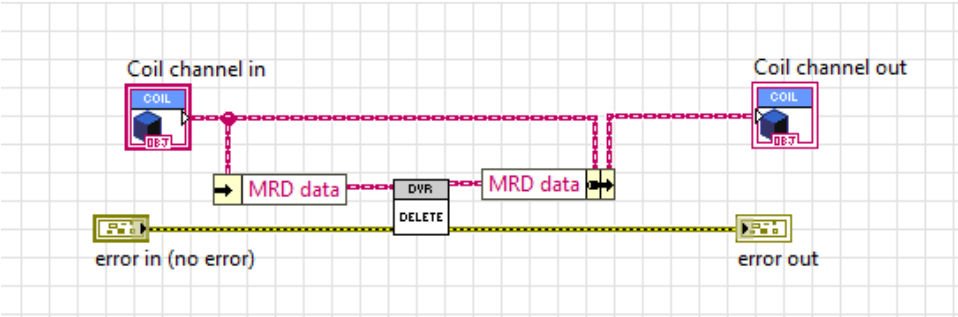


Figure 20: block diagram of cleanup data

This vi deletes the mrd data from the storage container to free up memory.

3.2.5. Amp

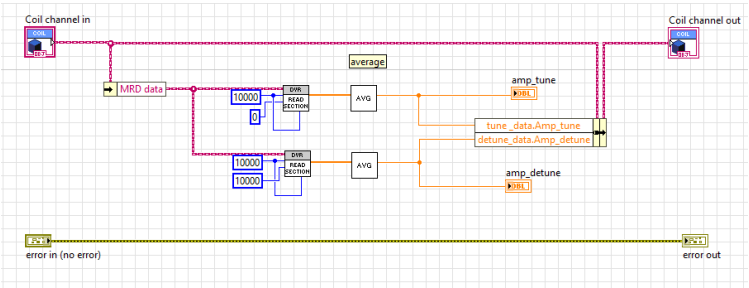


Figure 21: Block diagram of amp.vi

This vi calculates the average value for a data subset.
For tune 0- 10000 is used.
For detune 10000-19999 is used

Size of the arrays is 10000.

In the original script, arrays are defined from 1 – 10000. In LabVIEW: arrays start at ‘0’.

Average value

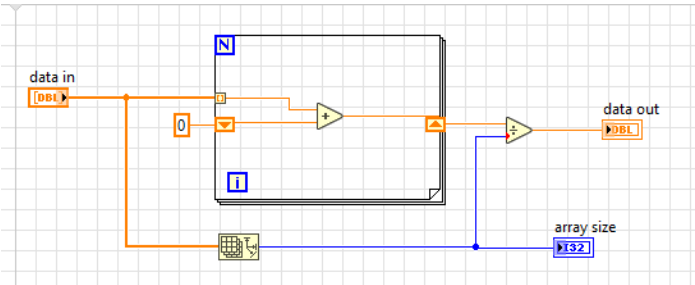


Figure 22: block diagram of average.vi

This particular implementation is chosen to remain true to the script. The LabVIEW mean calculation gives the same result.

3.2.6. Amp_dB

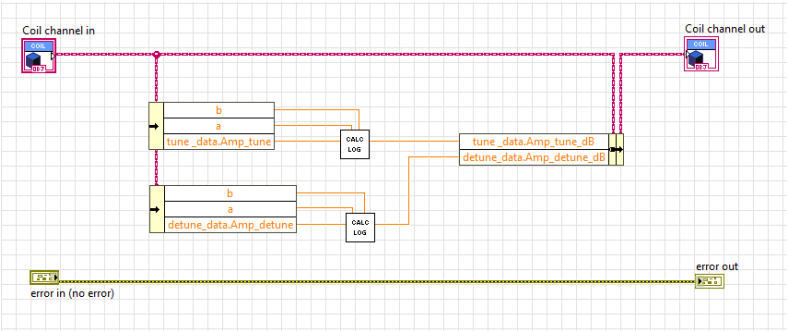


Figure 23: block diagram of amp_dB.vi

Calculate log

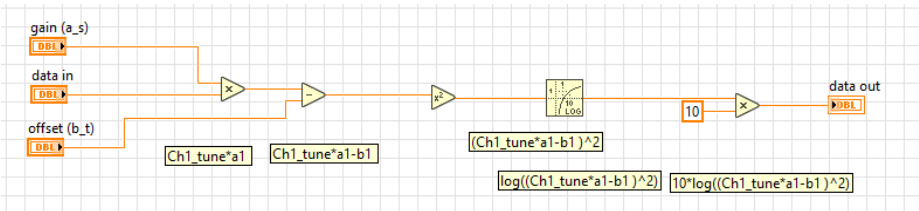


Figure 24: block diagram of calculate log

3.2.7. Mean

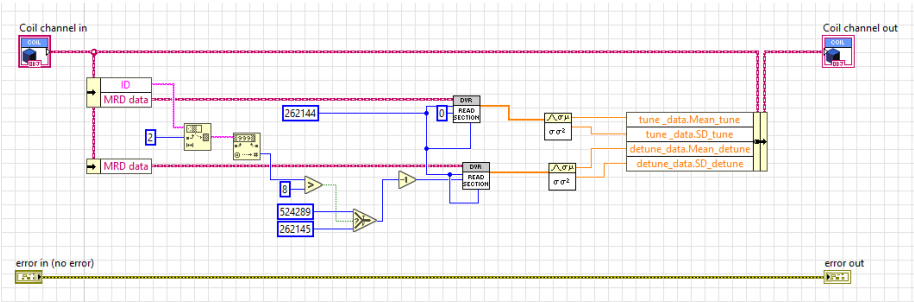


Figure 25: block diagram of mean.vi

This via calculates the mean and SD (standard deviation) for a data subset
The size of the data subset is the same for all channels: 262144
The start index different for channels 9 to 12.
For channel 1 to 8: 262144
For channel 9 to 12: 524288

In the original script, arrays are defined from 1 – 10000. In LabVIEW: arrays start at ‘0’.
The numbers in the LabVIEW code are the same as in the original script. ‘1’ is distracted to accommodate the different array definition in LabVIEW.

3.2.8. SD_dB

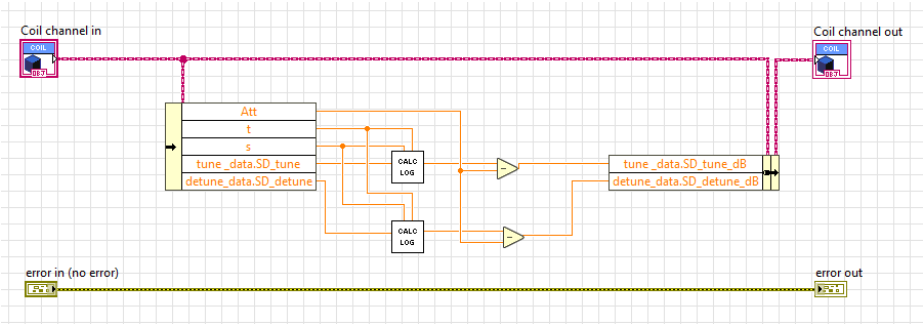


Figure 26: block diagram of SD_db.vi

This vi calculates the log from the SD values. For the log calculation see ‘Amp_db’ chapter.

3.2.9. M_SD

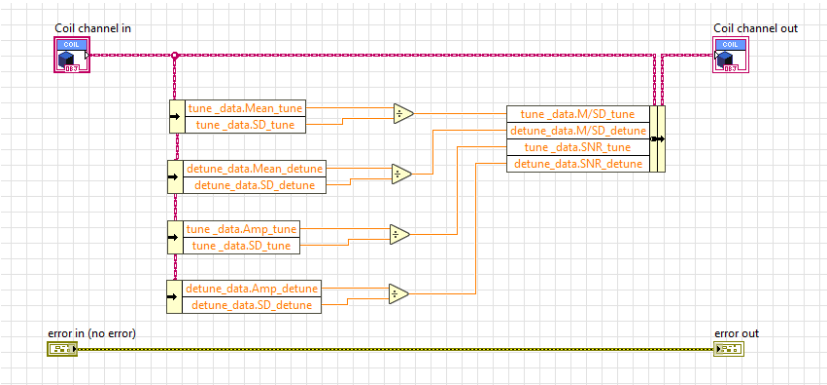


Figure 27: block diagram of M_SD.vi

This vi calculates mean / SD and Amp / SD.

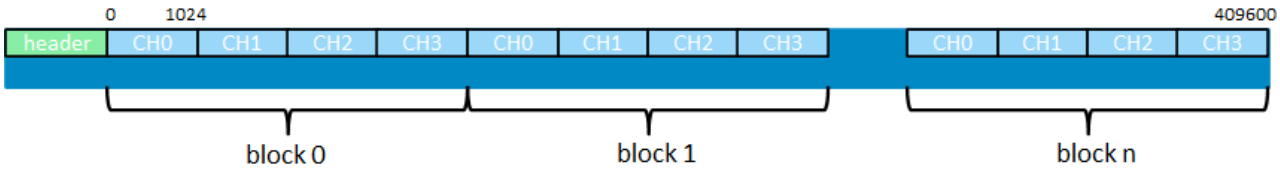
3.3. MRD file

3.3.1. MRD file specification

3.3.1.1. Introduction

The internal format of the mrd file consists of a header and data blocks. First there is a header with general file information. The data part is located after the header and consists of multiple blocks of data for a certain nr of measurement channels. Every channel consists of 1024 samples in every block.

There are 2 types of mrd files:
- pc_snr_1t5_*_*.MRD (containing 4 channels of measurement data)
- pc_noise_1t5_*.MRD (containing 12 channels of measurement data)



3.3.1.2. Header

- 128 bits (16 bytes) of non data
- 32 bits (4 bytes) contains the header size
- 32 bits (4 bytes) of zeroos (can be ignored)
- Header info:
  ID="pc_snr_1t5_4"N="4096"DATE="20-APR-2022\s09:33:54.66"TYPE="COMPLEX_DOUBLE"\00

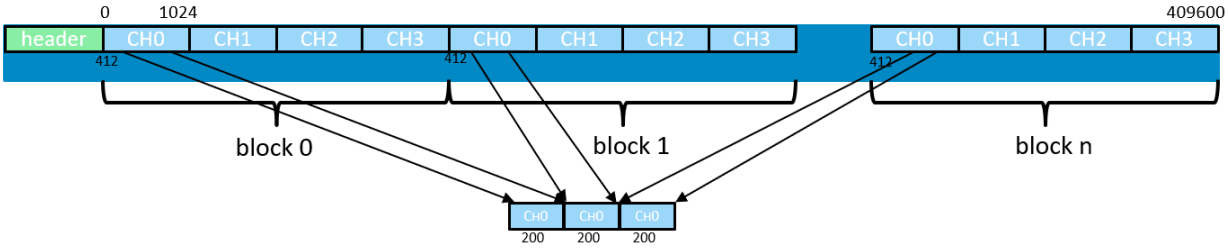
3.3.1.3. Data block

- 128 bits (16 bytes) of non data

- 32 bits (4 bytes): datasize1: number of real+imaginary samples
- Datasize2: is always ‘0’ (can be ignored)
- Array of actual data (alternated I & Q data)

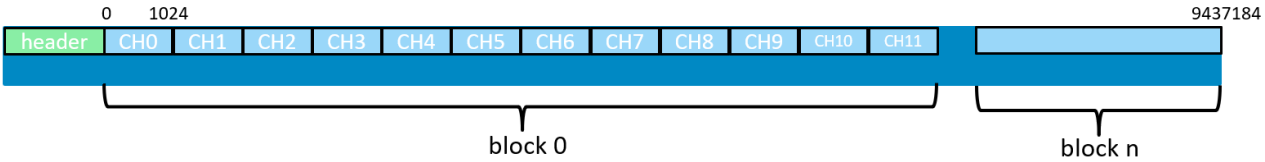
3.3.1.4. SNR file

From every data block, a nr of samples (200) is selected, starting from position 412. This data subset is put into an new data array for every measurement channel.



3.3.1.5. Noise file

In the noise file all 12 channels are present.



The data for each channel is read. The modulus (abolute value) is calculated for every imaginary value

3.3.2. Init

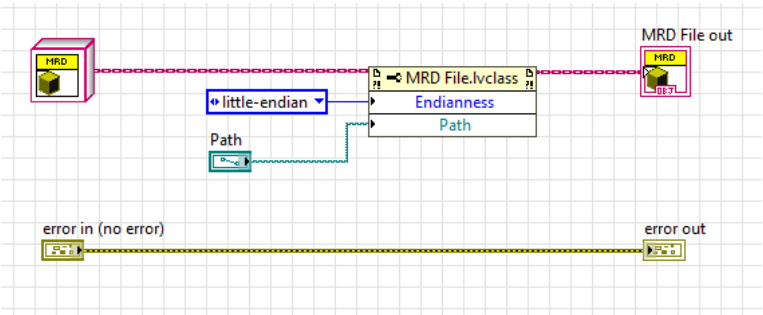
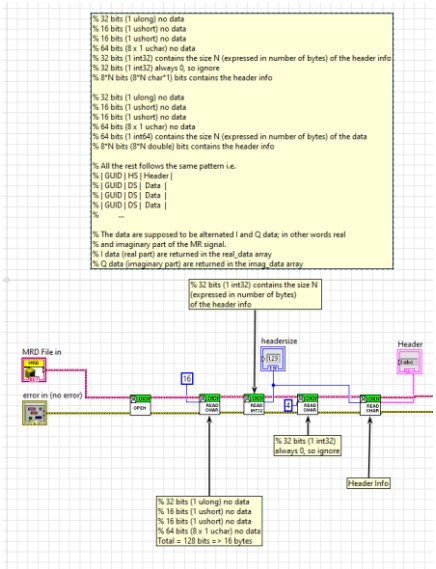


Figure 28: block diagram of init.vi

Store the file path and endian setting.

3.3.3. Read



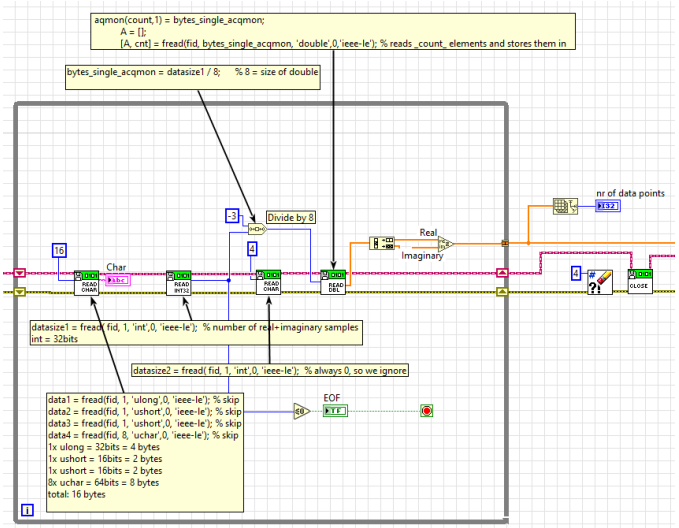


Figure 29: Block diagram of read.vi (part one) *Figure 30: block diagram of read.vi (part two)*

This vi read the entire mrd file, header and data. Reading of data is stopped when EOF (end of file) is reached (error 4).

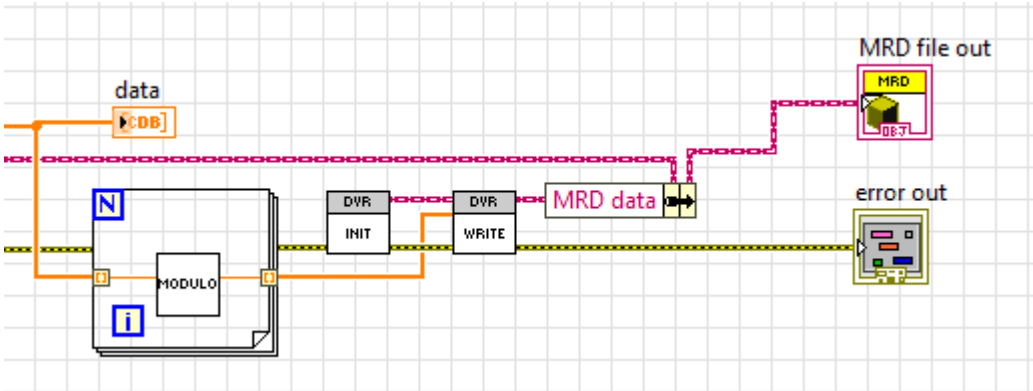


Figure 31: block diagram for read.vi (part three)

The real and imaginary data is transformed into a double array by calculating the modulus (abolute value) for every imaginary value shown below:

$$|Z| = \sqrt{a^2 + (ib)^2}$$

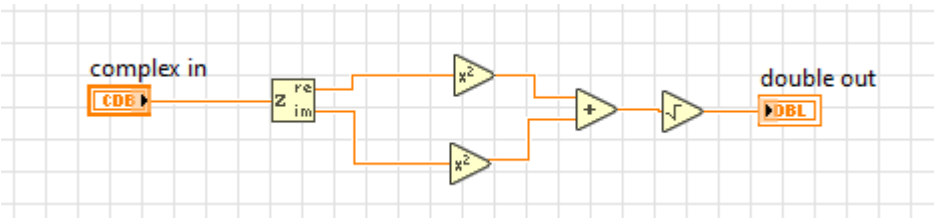


Figure 32: block diagram for modulus.vi

The data is stored in a storage container (MRD data).

3.4. MRD data

3.4.1. Init

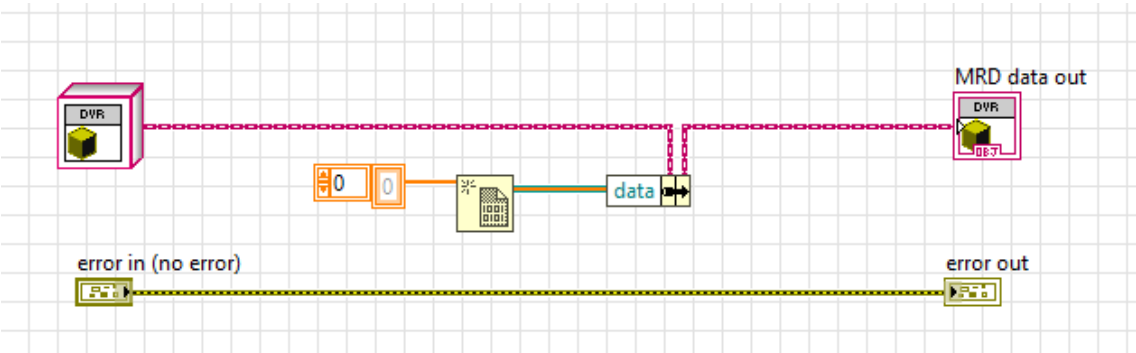


Figure 33: block diagram for init.vi

Initializes the DVR (data pointer).

3.4.2. Write

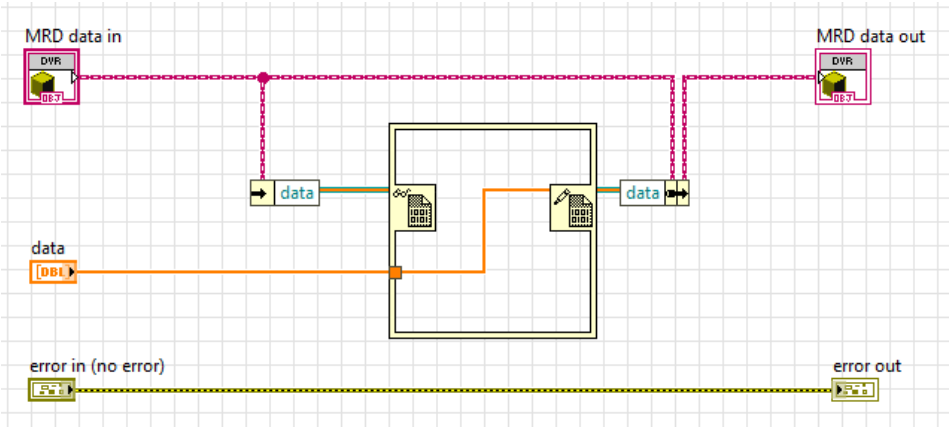


Figure 34: block diagram for write.vi

Write (all) data to the DVR.

3.4.3. Append

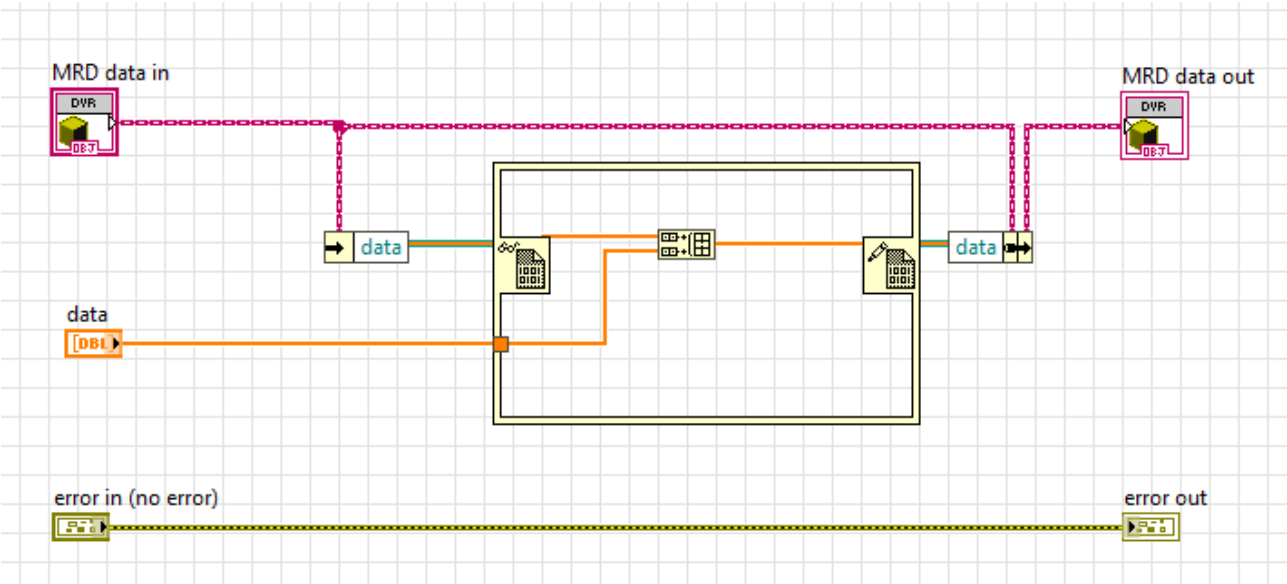


Figure 35: block diagram for append.vi

Appends data to the DVR array.

3.4.4. Read

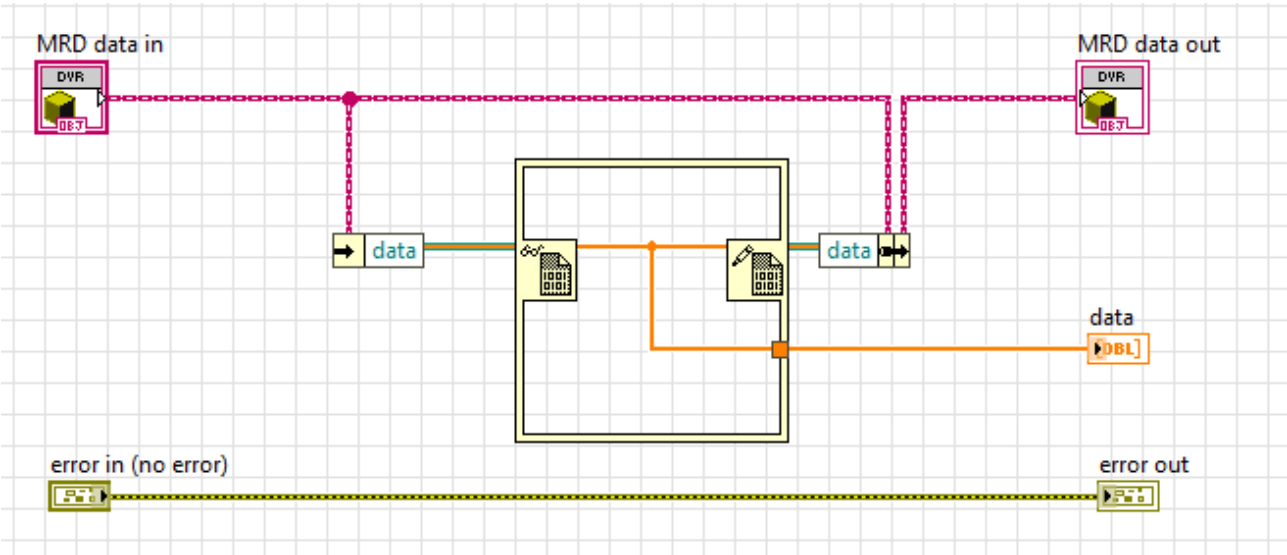


Figure 36:block diagram for read.vi

Reads all data from DVR.

3.4.5. Data size

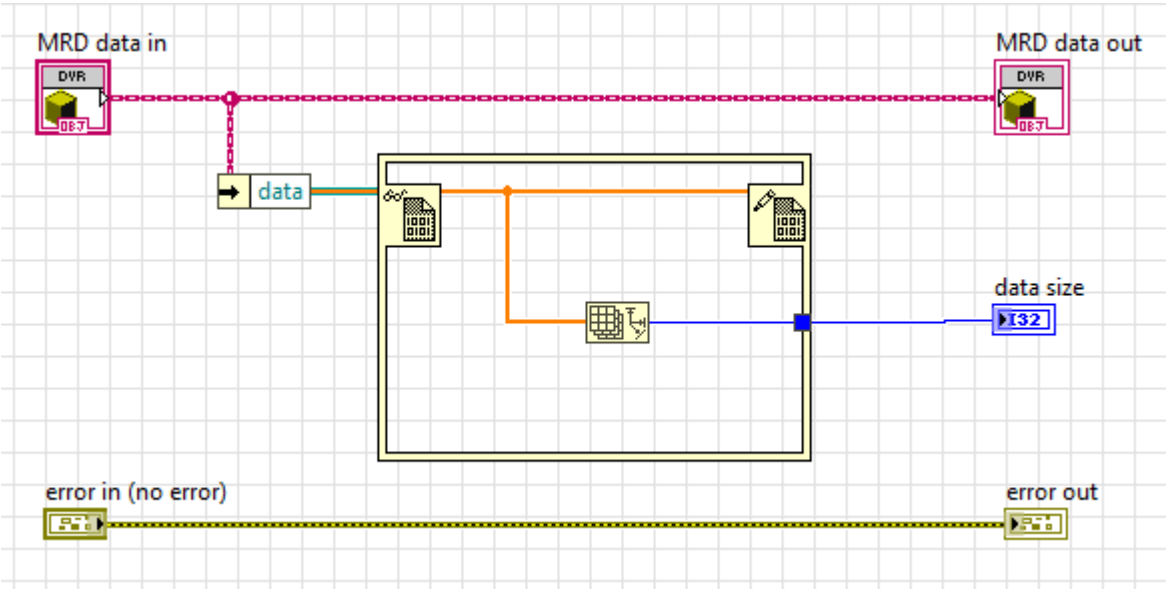


Figure 37: block diagram for data size.vi

Outputs the data size of the data array.

3.4.6. Delete

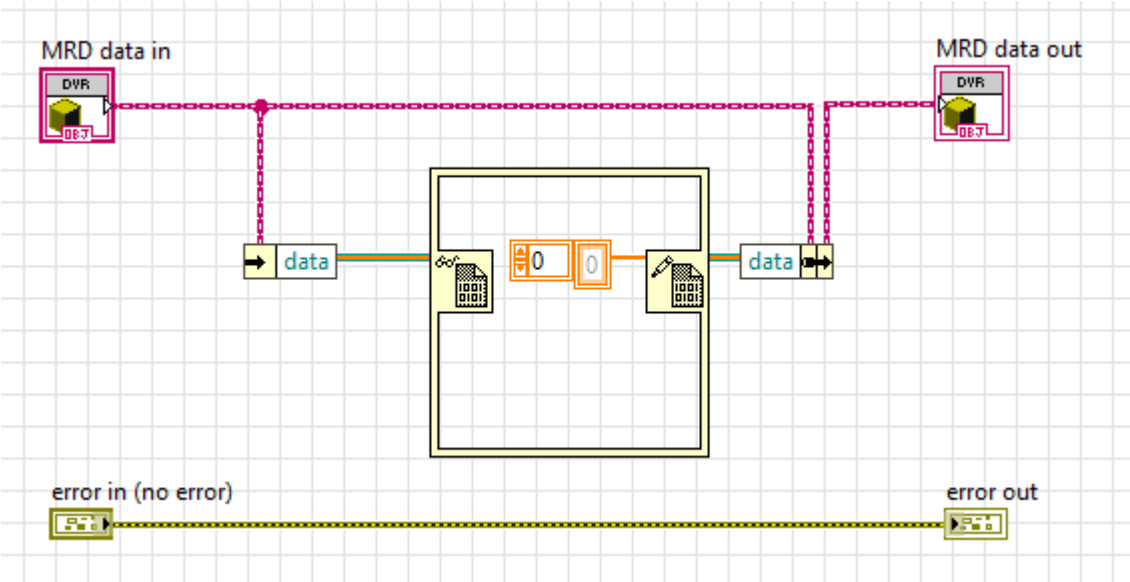


Figure 38: block diagram for delete.vi

Deletes all data from storage (replaces it with an empty array).

3.4.7. Read section

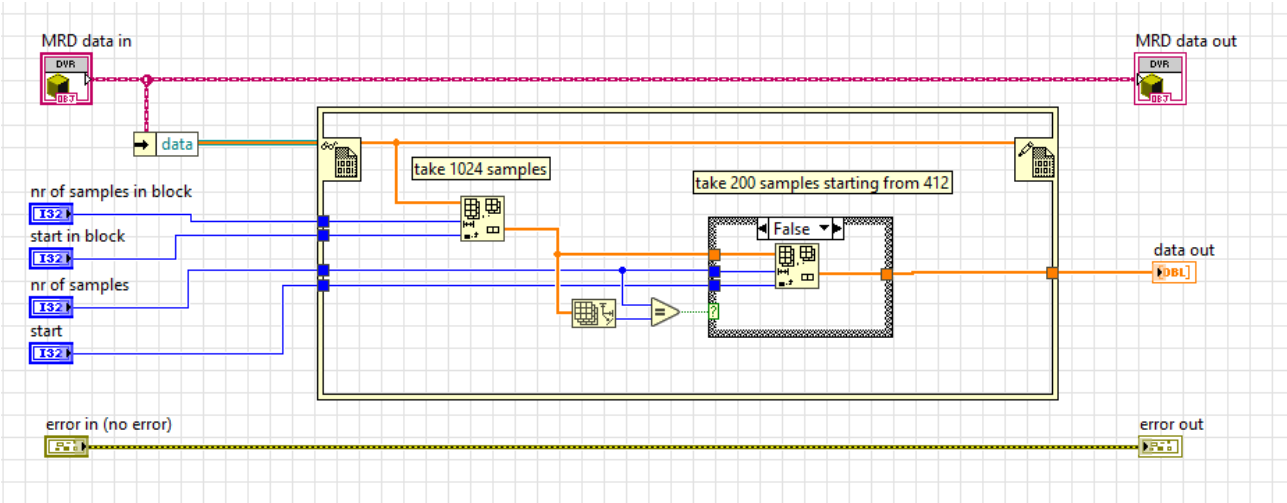


Figure 39: block diagram for read section.vi

This vi takes a certain section from the array (default 1024 samples) and from this sub section you can select again a subsection.

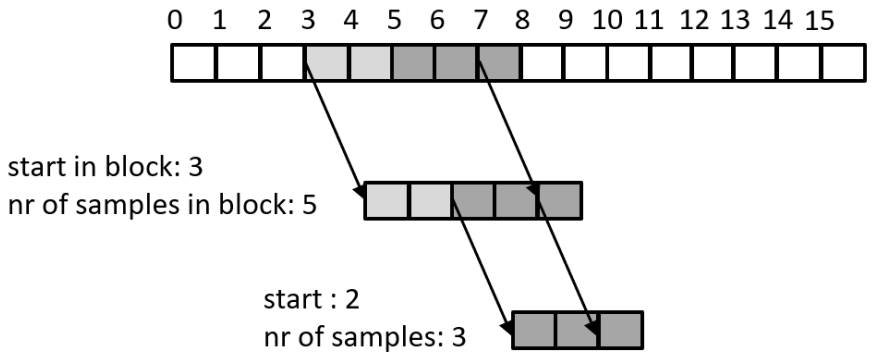


Figure 40: selection example

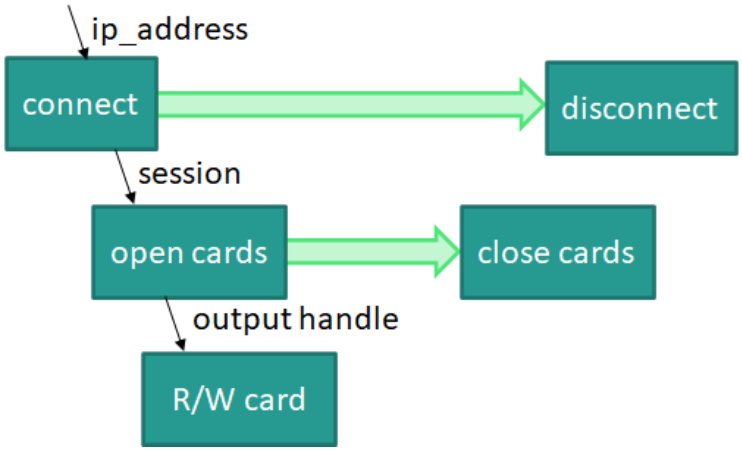
4. Pickering power and rf switcher

Figure 41: pickering 4 channel interface

The chassis has two cards installed. The power switcher and the rf channel switcher.

4.1. Architectcture

The driver architecture is such that the connect / disconnect layer and open cards / close cards layer function independently of which cards are installed. They generate the session handle and the output handles. The actual card can be driven easily using read and write methods. The required output handle for the card can be easly obtained from the storage container.



4.2. Example

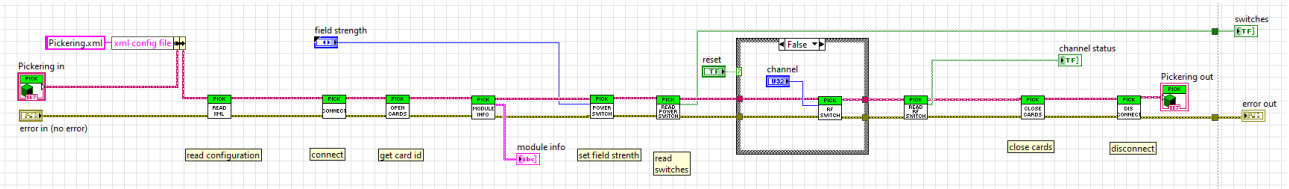


Figure 42: block diagram for example.vi

The example initializes the cards and the use can set the field strength of the power switch (1.5T or 3T and can select a RF channel. Or reset all rf channels in the 'off' state. At last it closes the cards.

The driver will read the following info from a xml configuration file.

4.3. Read xml config

This sub module reads the parameters from the xml configuration file and stores the data in a storage unit. The xml file is stored in the root of the project folder.

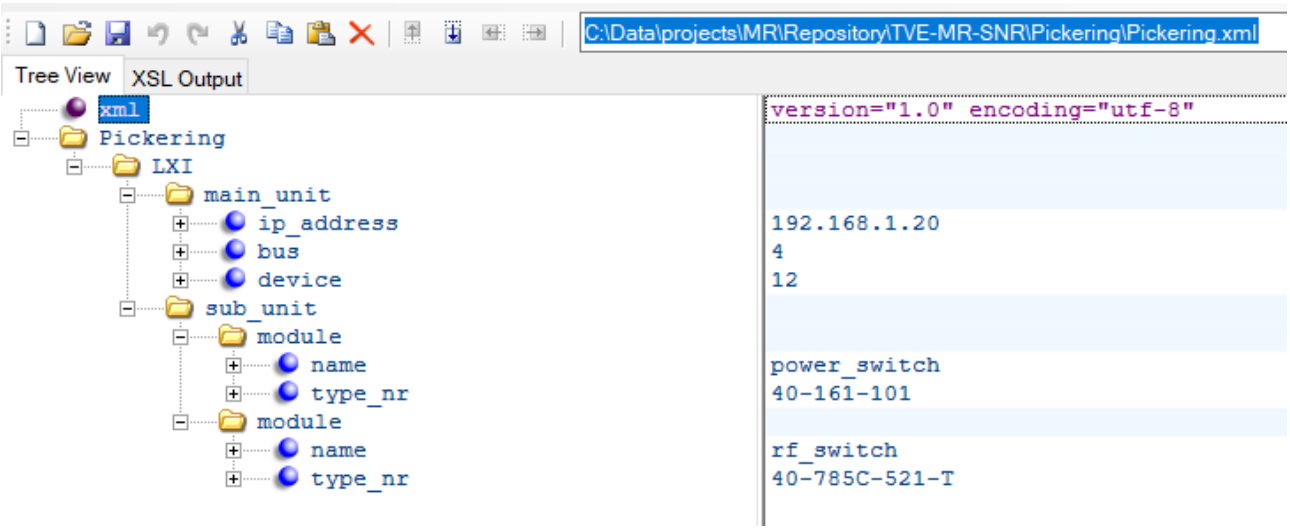


Figure 43: xml configuration file

nr	subsection	item	description
1	main_unit	ip_address	ip_address of the chassis
2	main_unit	bus	nr of slots in the main chassis
3	main_unit	device	
4	sub_unit	name	name of the card (rf_power_switch or rf_channel_switch)
5	sub_unit	type_nr	type_nr of the card

Based on the type_nr additional information will be stored in an internal memory storage module. The data is stored in a 2 level hierarchy.

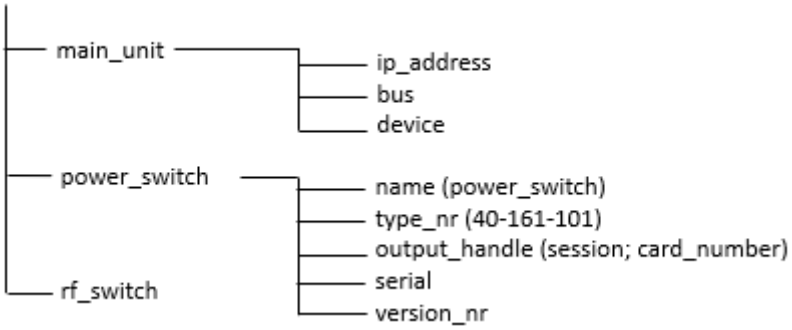


Figure 44: internal storage hierarchy

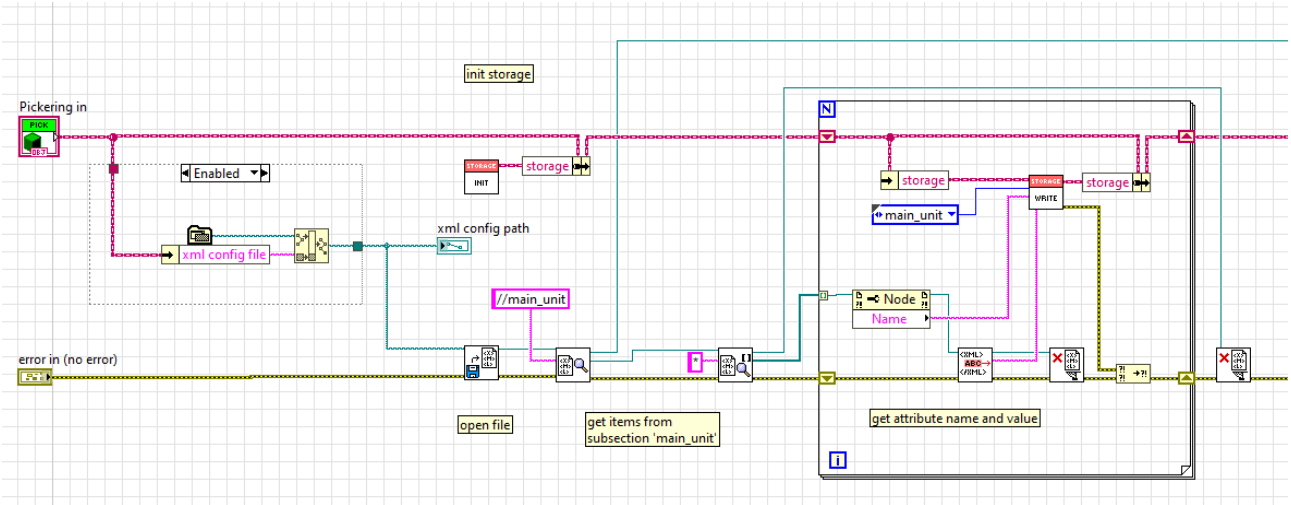


Figure 45: blockdiagram for read xml config (part1)

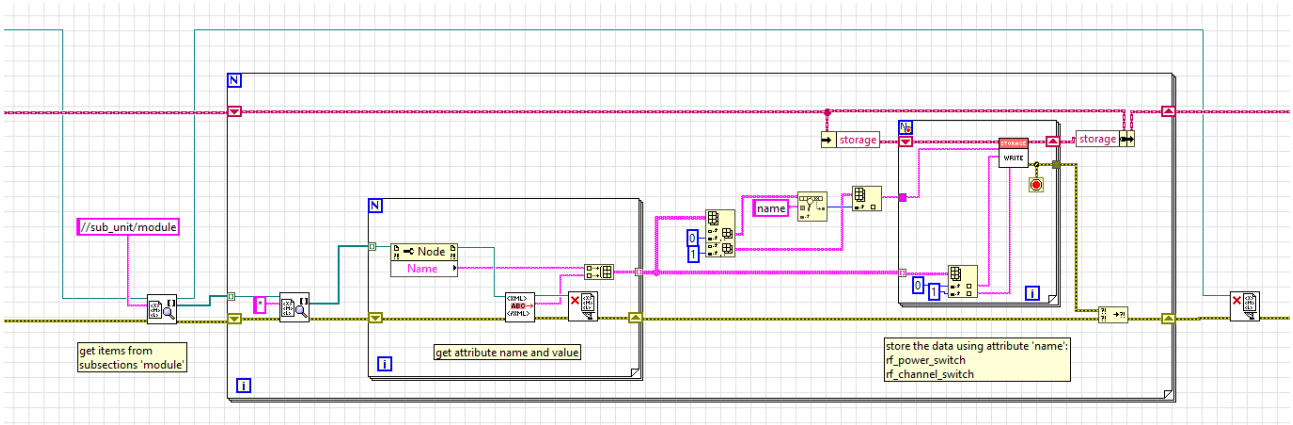


Figure 46: blockdiagram for read xml config (part2)

All cards which are present in the xml file are loaded into the storage container.

4.4. Connect

The driver uses sub vis's from the libraries: picmlx.llb and piplx.llb, These are installed when executing ClientBridge v1.65.0.0.exe, which can be downloaded here: https://downloads.pickeringtest.info/downloads/drivers/Sys60/?__hstc=256762388.eedfa2368f7e14e2b3d64ff5a33e66d7.1652176148929.1653387531301.1654685204078.5&__hssc=256762388.1.1654685204078&__hsfp=1677299015

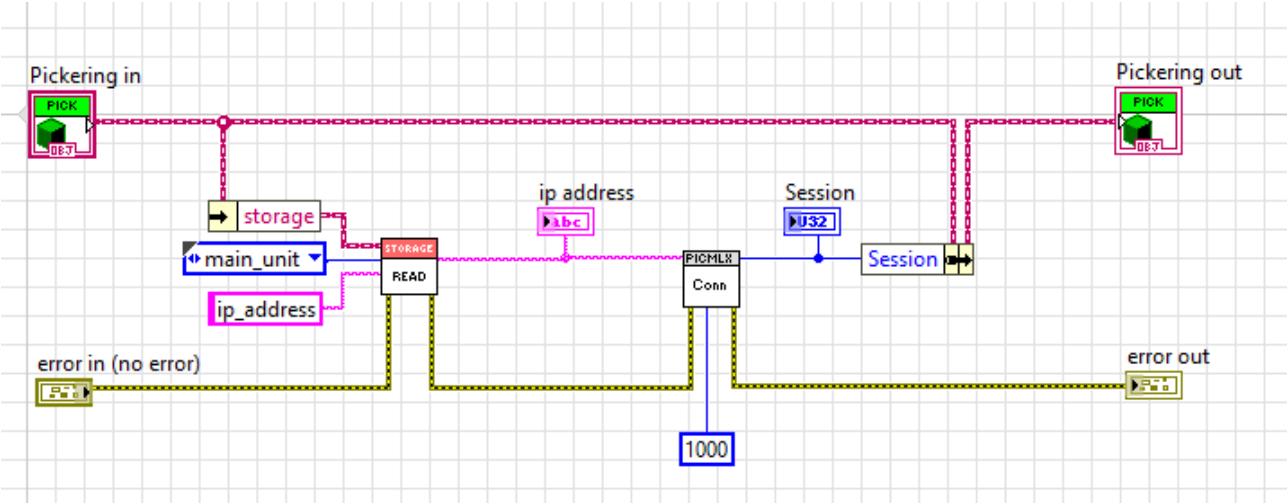


Figure 47: block diagram for connect.vi

4.5. Disconnect

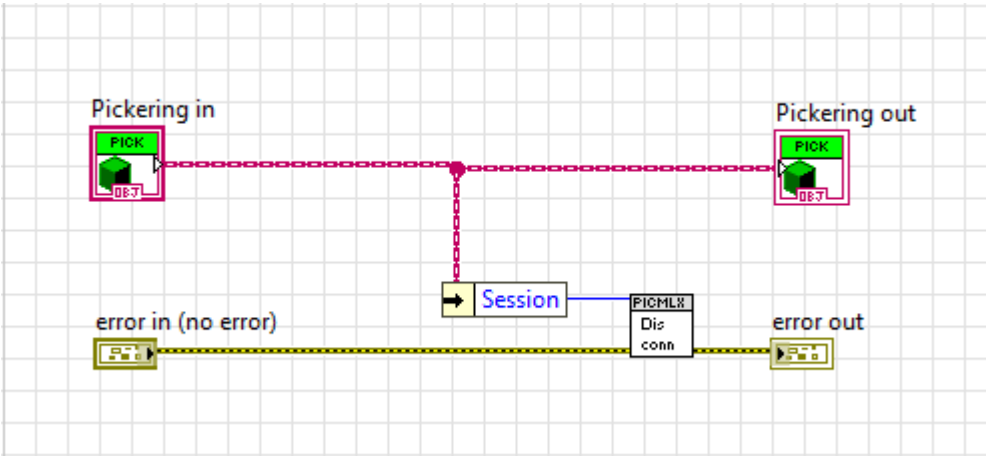


Figure 48: block diagram for disconnect,vi

4.6. Open cards

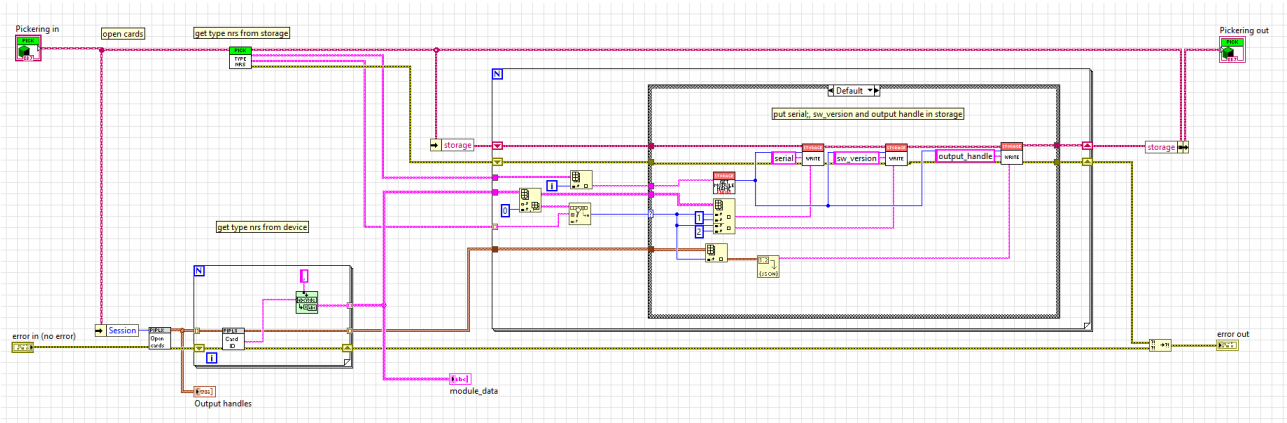


Figure 49: block diagram for open cards.vi

Card are initialized (opened). An output handle to each card is generated. The card information (type nr and sw version) is read from the cards. The type nr is used to link the cards (output handle) to the card name. All data is stored in the starage container. When a certain card is not available, it will be removed from the storage container. Output handle cluster is stored in json format.

4.7. Get type nrs from storage

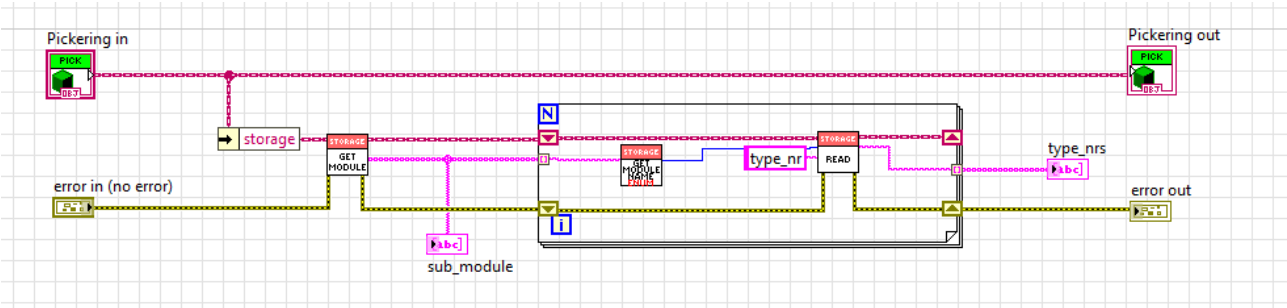


Figure 50: block diagram for get type nrs from storage.vi

This sub vi is used by ‘open cards.vi’ to get the type nrs from the storage container.

4.8. Close cards

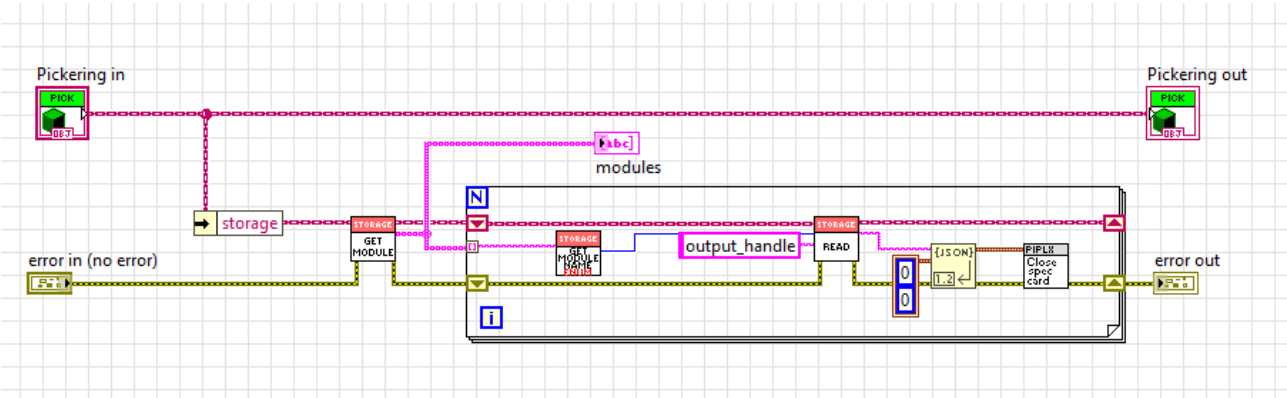


Figure 51: block diagram for close cards.vi

4.9. Get module info

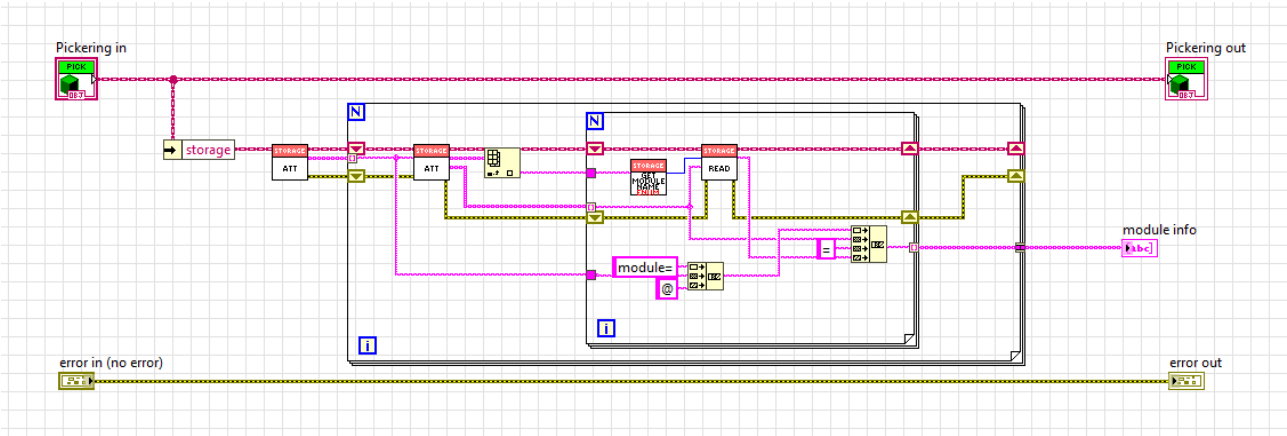


Figure 52: block diagram for get module info

This vi will output all stored modules info from the storage container.

Figure 53: frontpanel of get module info.vi

Module name is separated from the data using '@'.

4.10. Write power switch

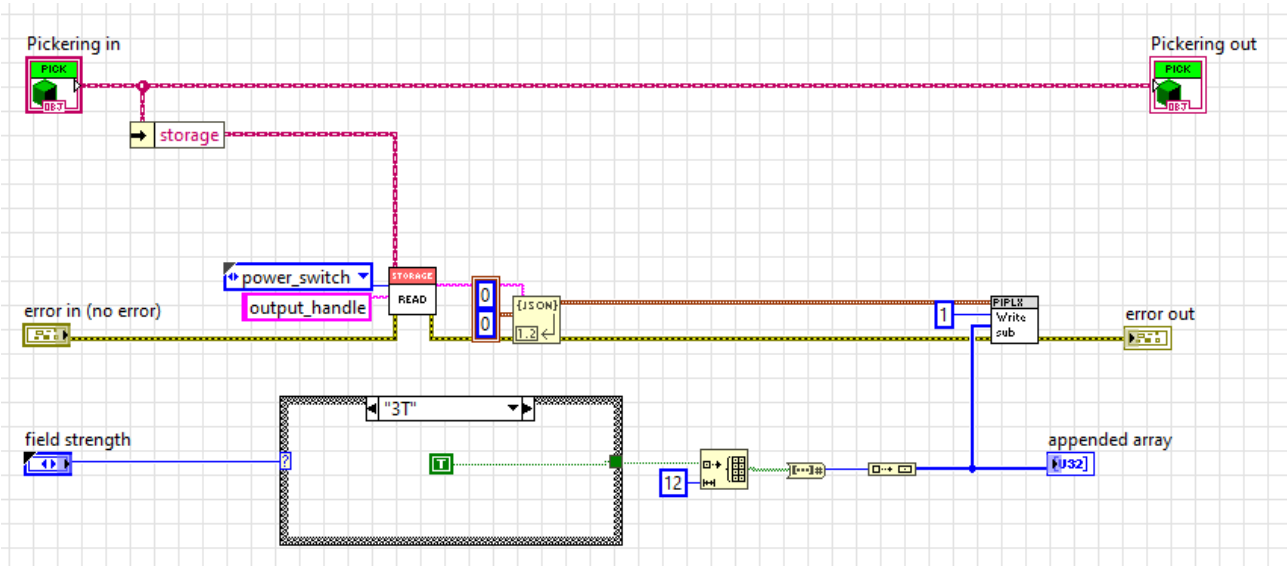


Figure 54: block diagram for write power switch.vi

This vi writes all switches to 'on' for 3T and all to 'off'in case of 1.5T.

4.11. Read power switch

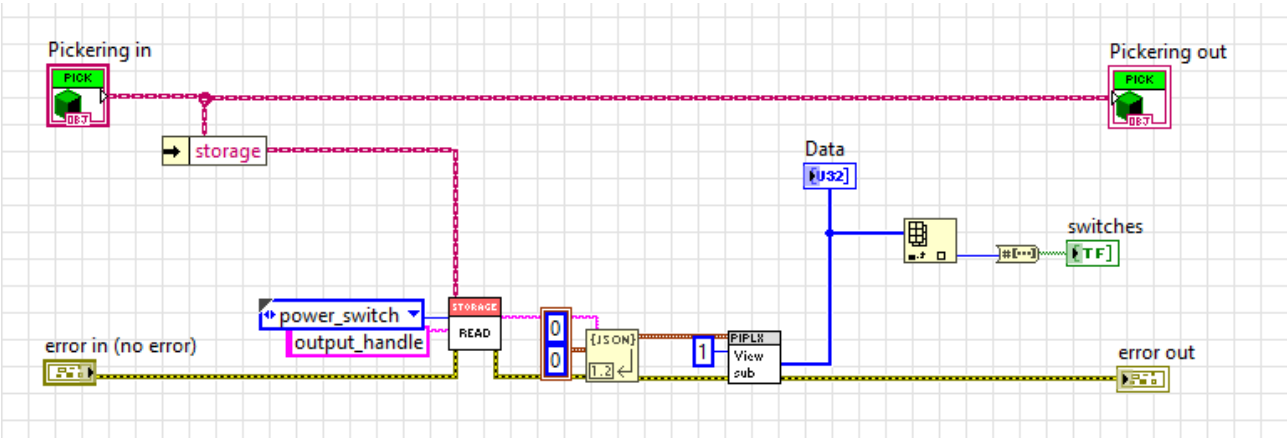


Figure 55: block diagram for read power switch.vi

This vi can be used to read out the status of the switches.

4.12. Write rf switch

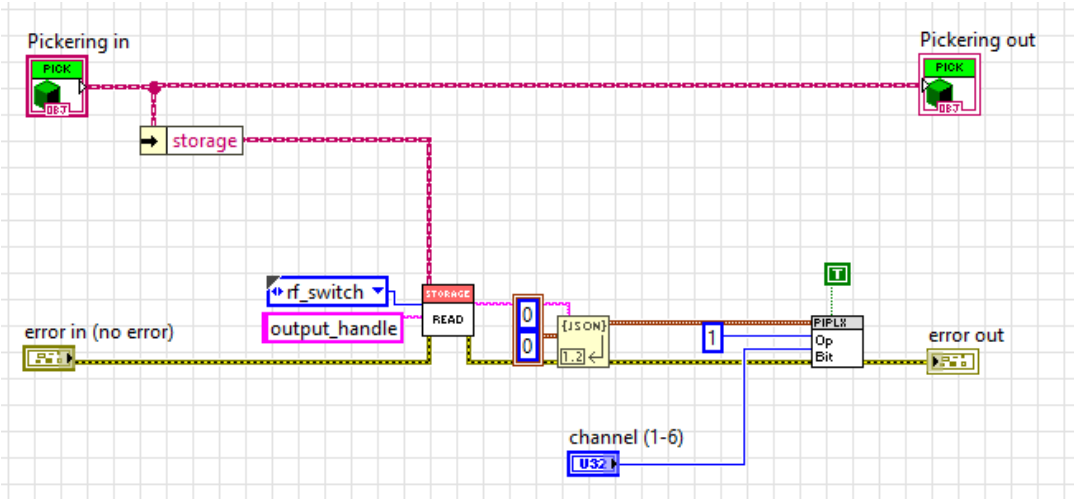


Figure 56: block diagram for write rf switch.vi

This vi sets the selected switch to ‘on’, any other switch is automatically set to ‘off’.

4.13. Read rf switch

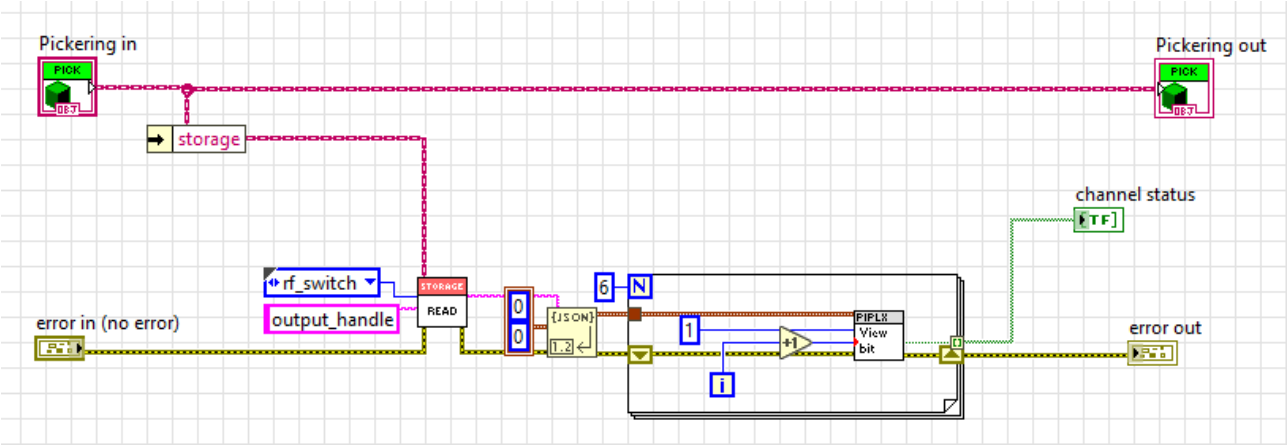


Figure 57: block diagram for read rf swtich.vi

This vi can be used to read out the status of the switches.

4.14. Reset rf switch

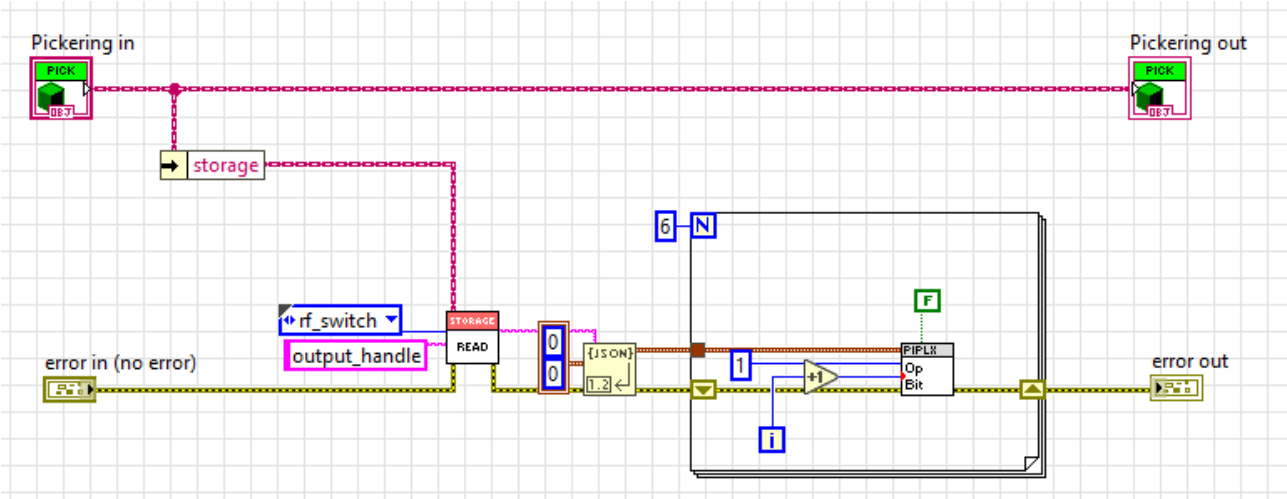


Figure 58: block diagram for reset rf switch.vi

This vi resets the switches: putting them in the ‘off’state.

<Always put the Philips shield at the bottom of the last page of the document.
To meet the Philips branding requirements, and to prevent the shield from wandering around, the Philips shield has been put in a 3-row table with fixed row heights.
Keep the Philips shield and the proprietary statement together!>

Philips' proprietary information. Unauthorized use is prohibited.

<DELETE THIS PAGE BEFORE FINALIZING THE DOCUMENT CREATED FROM THIS FORM>

<This page is NOT part of the controlled document you create from this form. This section contains information on the History and Control of the form.>

Control of the Form

Approver
GEN: Process Owner
Q&R: Q&R Manager

(approval via electronic signature in document management tool)

History of the Form

Revision	Doc. Date	Author	CR-ID	Description
01	2018 Oct 22	Rene Vermeeren	CR2018/097	Adapted to new format Added to be PDLM compliant: • Information Security, Privacy and Product and Service Security as possible topics in chapter 7.2 • Moved SW driven warning messages from 7.6 to 7.1.1 • Experienced Design in chapter 7.7